МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Государственный Университет Управления

Кафедра: Институт Информационных Систем

Лабораторная работа № 5

Хеширование и экспериментальное исследование методов разрешения коллизий

Дисциплина: Теория алгоритмов

Направление подготовки: 38.03.05 «Бизнес-информатика»

Язык реализации: Python 3.10+

Вариант: 14

Выполнил: студент группы БИ 2-1 Лапшенков Григорий Витальевич

Проверил: [должность, ФИО преподавателя]

[Город], 2026

1. Цель работы и задание варианта

1.1. Цель работы

Изучить принципы хеширования как способа организации данных с временем доступа $O(1)$ в среднем, реализовать хеш-таблицу и экспериментально исследовать влияние хеш-функции и коэффициента заполнения на её производительность.

1.2. Задание варианта 14

Таблица 1. Исходные данные варианта

Параметр	Значение
Тип ключей	целые случайные, равномерно из $[0;\ 10^7)$, без повторов
Хеш-функция	У — метод умножения
Разрешение коллизий	Ц — метод цепочек; К — квадратичное пробирование
Число элементов n	25 000
Размер таблицы m	32 768 ($= 2^{15}$)
Коэффициент заполнения	$\alpha = n/m = 0,7629$
Дополнительное задание	В7 — подсчёт частот в логе из 10^6 записей

Необходимо: реализовать таблицу двумя способами (методы `insert`, `search`, `delete` со счётчиком проб), измерить время вставки и поиска, среднее число проб и распределение длин цепочек; построить зависимость числа проб от α с теоретической кривой; сравнить качественную и заведомо неудачную хеш-функции; сопоставить реализацию со встроенным `dict`; решить задание В7.

2. Хеш-функция и способы разрешения коллизий

2.1. Метод умножения

$$h(k) = \lfloor m \cdot \{k \cdot A\} \rfloor, \qquad A = \frac{\sqrt{5}-1}{2} \approx 0{,}6180339887,$$

где $\{x\}$ — дробная часть. Метод зависит от всех разрядов ключа, и размер таблицы может быть произвольным — в том числе степенью двойки, как в варианте ($m = 2^{15}$), тогда как для метода деления это недопустимо. Пример: $k = 1234567$: $k \cdot A = 763004{,}3674$, $\{kA\} \approx 0{,}3674$, $h(k) = \lfloor 32768 \cdot \{kA\} \rfloor = 12038$.

В качестве заведомо неудачного примера для сравнения используется $h(k) = k \bmod 10$: функция принимает лишь 10 различных значений, поэтому почти вся таблица остаётся пустой, а все ключи концентрируются в десяти ячейках.

2.2. Метод цепочек

Элементы с одинаковым значением хеша хранятся в списке, присоединённом к ячейке. Проба — просмотр одного элемента цепочки. Ожидаемое число проб при $\alpha = 0,763$:

- успешный поиск: $1 + \frac{\alpha}{2} \approx 1,38$;
- неуспешный поиск: $\alpha \approx 0,76$ (просматривается вся цепочка, в среднем длины α).

Трудоёмкость $\Theta(1+\alpha)$ в среднем и $\Theta(n)$ в худшем случае.

2.3. Квадратичное пробирование

Элементы хранятся в самом массиве; при занятости ячейки проверяется следующая по правилу

$$h_i(k) = \left(h(k) + c_1 i + c_2 i^2\right) \bmod m, \qquad c_1 = c_2 = \tfrac{1}{2} \ \Rightarrow\ h_i(k) = \left(h(k) + \frac{i(i+1)}{2}\right) \bmod m.$$

Выбор коэффициентов важен: при $m = 2^s$ последовательность $\frac{i(i+1)}{2} \bmod m$, $i = 0, \dots, m-1$, проходит **все** ячейки таблицы, поэтому вставка всегда находит свободное место при $\alpha < 1$. Распространённый выбор $c_1 = c_2 = 1$ при таком m непригоден: $i^2 + i$ всегда чётно, и достигается лишь половина ячеек. Покрытие всей таблицы проверяется в программе (функция `self_test`). Соседние пробы вычисляются инкрементально: $pos_i = pos_{i-1} + i$.

Первичной кластеризации (как при линейном пробировании) нет, но ключи с одинаковым начальным значением $h(k)$ проходят одну и ту же последовательность — это **вторичная кластеризация**, из-за которой число проб несколько выше, чем при идеальном равномерном хешировании:

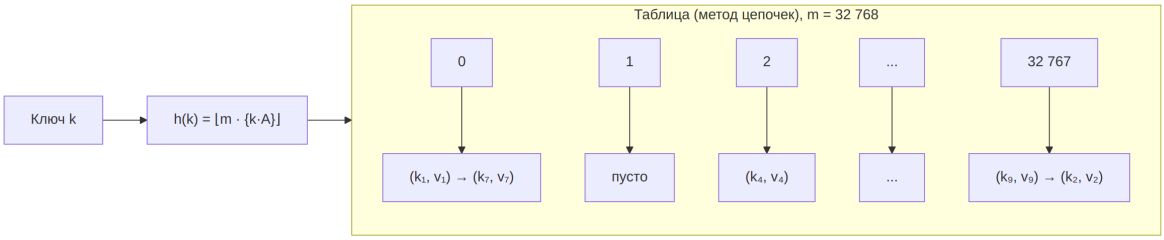
- успешный поиск: $\approx \frac{1}{\alpha} \ln \frac{1}{1-\alpha} \approx 1,89$;
- неуспешный поиск: $\approx \frac{1}{1-\alpha} \approx 4,22$.

Эти формулы — оценка для равномерного хеширования; для квадратичного пробирования ожидаются несколько большие значения.

Удаление выполняется постановкой метки «удалено» (tombstone): простое обнуление ячейки разорвало бы последовательность проб других ключей. При вставке запоминается первая встреченная метка, но поиск продолжается до пустой ячейки, чтобы не создать дубликат уже существующего ключа.

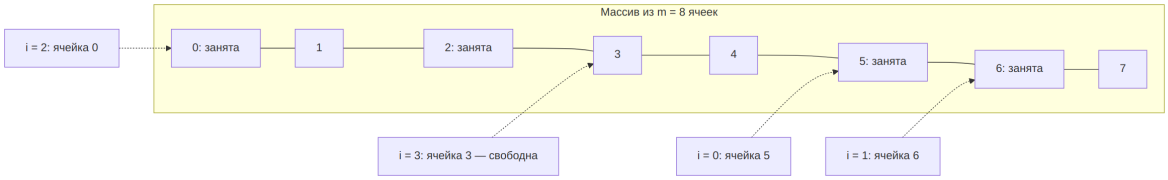
3. Схемы структуры данных и блок-схема

3.1. Схема таблицы при методе цепочек

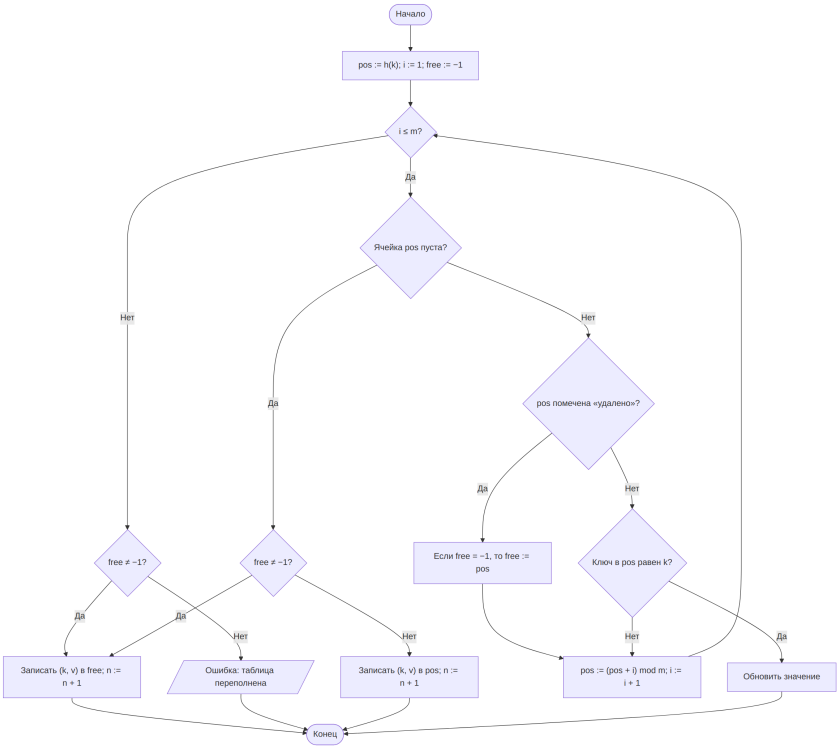


3.2. Схема проб при квадратичном пробировании

Пример для $m = 8$ и ключа с $h(k) = 5$: проверяются ячейки $5,\ 6,\ 0,\ 3,\ \dots$, то есть $(5 + 0) \bmod 8$, $(5 + 1) \bmod 8$, $(5 + 3) \bmod 8$, $(5 + 6) \bmod 8$.



3.3. Блок-схема вставки при квадратичном пробировании



4. Листинг программы

Программа содержит хеш-функции, обе таблицы (`insert`, `search`, `delete`, счётчик проб), контроль корректности (случайные операции сверяются со словарём `dict`), замеры, решение задания B7 и построение графиков.

```
"""
Лабораторная работа № 5. Вариант 14.
Хеш-таблица: метод умножения; разрешение коллизий цепочками и квадратичным
пробированием. Целые случайные ключи, n = 25 000, m = 32 768.
Дополнительное задание B7: подсчёт частот в логе из 10^6 записей.
"""
import gc
import heapq
import json
import math
import random
import statistics
import time
from collections import Counter

M = 32_768                    # размер таблицы (по варианту)
N = 25_000                    # число хранимых ключей
KEY_RANGE = 10_000_000        # ключи — случайные целые из [0; 10^7)
A = (math.sqrt(5) - 1) / 2    # постоянная Кнута, A ≈ 0,6180339887
QUERIES = 10_000              # число запросов каждого вида
REPEATS = 5                   # повторов замера (берётся медиана)


# ---------------------------------------------------------- хеш-функции
def hash_mul(key, m):
    """Метод умножения: h(k) = floor(m * {k * A})."""
    return int(m * ((key * A) % 1.0))
```

```python
def hash_bad(key, m):
    """Заведомо неудачная функция: всего 10 различных значений."""
    return key % 10


# ------------------------------------------------ метод цепочек
class ChainHashTable:
    """Хеш-таблица с разрешением коллизий методом цепочек.
    Проба — просмотр одного элемента цепочки (сравнение ключей)."""

    def __init__(self, m=M, hash_func=hash_mul):
        self.m, self.hash_func = m, hash_func
        self.slots = [[] for _ in range(m)]
        self.n = 0
        self.probes = 0

    def insert(self, key, value):
        chain = self.slots[self.hash_func(key, self.m)]
        for item in chain:
            self.probes += 1
            if item[0] == key:
                item[1] = value
                return
        chain.append([key, value])
        self.n += 1

    def add(self, key, delta=1):
        """Увеличить значение по ключу (счётчик); при отсутствии ключа — создать."""
        chain = self.slots[self.hash_func(key, self.m)]
        for item in chain:
            self.probes += 1
            if item[0] == key:
                item[1] += delta
                return
        chain.append([key, delta])
        self.n += 1

    def search(self, key):
        for item in self.slots[self.hash_func(key, self.m)]:
            self.probes += 1
            if item[0] == key:
                return item[1]
        return None

    def delete(self, key):
        chain = self.slots[self.hash_func(key, self.m)]
        for i, item in enumerate(chain):
            self.probes += 1
            if item[0] == key:
                del chain[i]
                self.n -= 1
                return True
        return False

    def items(self):
        return [(k, v) for chain in self.slots for k, v in chain]

    def chain_lengths(self):
        return [len(c) for c in self.slots]
```

```python
# ---------------------------------------- квадратичное пробирование
class QuadHashTable:
    """Хеш-таблица с открытой адресацией и квадратичным пробированием
    h_i(k) = (h(k) + i/2 + i^2/2) mod m = (h(k) + i(i+1)/2) mod m.
    При m = 2^s такая последовательность проходит все ячейки таблицы.
    Удаление — с пометкой DELETED (tombstone)."""

    EMPTY = object()
    DELETED = object()

    def __init__(self, m=M, hash_func=hash_mul):
        self.m, self.hash_func = m, hash_func
        self.keys = [self.EMPTY] * m
        self.values = [None] * m
        self.n = 0
        self.probes = 0

    def _locate(self, key):
        """Возвращает (позиция ключа или -1, позиция для вставки или -1)."""
        m, keys = self.m, self.keys
        pos = self.hash_func(key, m)
        first_free = -1
        for i in range(1, m + 1):
            self.probes += 1
            k = keys[pos]
            if k is self.EMPTY:
                return -1, (first_free if first_free != -1 else pos)
            if k is self.DELETED:
                if first_free == -1:
                    first_free = pos
            elif k == key:
                return pos, -1
            pos = (pos + i) % m            # следующий член: h + i(i+1)/2
        return -1, first_free              # просмотрена вся таблица

    def _store(self, key, value, free):
        if free == -1:
            raise RuntimeError("Таблица переполнена")
        self.keys[free], self.values[free] = key, value
        self.n += 1

    def insert(self, key, value):
        pos, free = self._locate(key)
        if pos != -1:
            self.values[pos] = value
        else:
            self._store(key, value, free)

    def add(self, key, delta=1):
        pos, free = self._locate(key)
        if pos != -1:
            self.values[pos] += delta
        else:
            self._store(key, delta, free)

    def search(self, key):
        pos, _ = self._locate(key)
        return self.values[pos] if pos != -1 else None
```

```
    def delete(self, key):
        pos, _ = self._locate(key)
        if pos == -1:
            return False
        self.keys[pos], self.values[pos] = self.DELETED, None
        self.n -= 1
        return True

    def items(self):
        return [(k, v) for k, v in zip(self.keys, self.values)
                if k is not self.EMPTY and k is not self.DELETED]


# ------------------------------------------------------------ проверка
def self_test():
    """Контроль корректности: случайные операции сверяются со словарём dict."""
    probe_cells = {(i * (i + 1) // 2) % M for i in range(M)}
    assert len(probe_cells) == M, "последовательность проб не покрывает таблицу"
    rng = random.Random(1)
    for cls in (ChainHashTable, QuadHashTable):
        t, ref = cls(64), {}
        for _ in range(20_000):
            key, op = rng.randrange(40), rng.random()
            if op < 0.4:
                t.insert(key, key * 2)
                ref[key] = key * 2
            elif op < 0.7:
                assert t.delete(key) == (key in ref)
                ref.pop(key, None)
            else:
                assert t.search(key) == ref.get(key)
        assert t.n == len(ref) and dict(t.items()) == ref
    print("Самопроверка пройдена: таблицы согласованы с dict, пробирование покрывает все ячейки.")


# ------------------------------------------------------------ замеры
def make_keys(n, rng):
    keys = rng.sample(range(KEY_RANGE), n)
    present = set(keys)
    found = [rng.choice(keys) for _ in range(QUERIES)]
    absent = []
    while len(absent) < QUERIES:
        q = rng.randrange(KEY_RANGE)
        if q not in present:
            absent.append(q)
    return keys, found, absent


def timed_pass(func, queries):
    gc.collect()
    t0 = time.perf_counter()
    for q in queries:
        func(q)
    return time.perf_counter() - t0


def bench_table(factory, keys, found, absent, repeats=REPEATS):
    """Время вставки/поиска и среднее число проб для одной реализации."""
    t_ins = []
```

```
    for r in range(repeats):
        table = factory()
        gc.collect()
        t0 = time.perf_counter()
        for k in keys:
            table.insert(k, k)
        t_ins.append(time.perf_counter() - t0)
        if r == 0:
            ins_probes = table.probes / len(keys)
    out = {"t_insert": statistics.median(t_ins), "p_insert": ins_probes}
    for label, qs in (("found", found), ("absent", absent)):
        table.probes = 0
        times = [timed_pass(table.search, qs) for _ in range(repeats)]
        out["t_" + label] = statistics.median(times)
        out["p_" + label] = table.probes / (len(qs) * repeats)
    return out, table


def bench_dict(keys, found, absent, repeats=REPEATS):
    t_ins = []
    for _ in range(repeats):
        d = {}
        gc.collect()
        t0 = time.perf_counter()
        for k in keys:
            d[k] = k
        t_ins.append(time.perf_counter() - t0)
    return {"t_insert": statistics.median(t_ins),
            "t_found": statistics.median(timed_pass(d.get, found) for _ in range(repeats)),
            "t_absent": statistics.median(timed_pass(d.get, absent) for _ in range(repeats))}


def theory_probes(alpha):
    """Теоретические оценки числа проб (равномерное хеширование)."""
    return {"chain_found": 1 + alpha / 2, "chain_absent": alpha,
            "quad_found": math.log(1 / (1 - alpha)) / alpha,
            "quad_absent": 1 / (1 - alpha)}


# ------------------------------------------------ задание B7 (лог)
def make_log(total, clients, rng):
    """Лог операций: запись = идентификатор клиента. Активность клиентов
    неравномерна (распределение Ципфа: вес клиента с рангом r ~ 1/r)."""
    ids = rng.sample(range(1_000_000, 10_000_000), clients)
    weights = [1 / r for r in range(1, clients + 1)]
    return rng.choices(ids, weights=weights, k=total)


def count_list(log):
    """Решение «через список и in»: два параллельных списка."""
    seen, counts = [], []
    for c in log:
        if c in seen:                       # O(u): линейный просмотр списка
            counts[seen.index(c)] += 1      # ещё один просмотр
        else:
            seen.append(c)
            counts.append(1)
    return dict(zip(seen, counts))
```

```
def count_hash(log, table_cls, m):
    table = table_cls(m)
    for c in log:
        table.add(c)
    return dict(table.items())


def count_dict(log):
    d = {}
    for c in log:
        d[c] = d.get(c, 0) + 1
    return d


def count_counter(log):
    return Counter(log)


def top10(freq):
    """10 клиентов с наибольшим числом операций (при равенстве — по возрастанию id)."""
    return heapq.nlargest(10, freq.items(), key=lambda kv: (kv[1], -kv[0]))


# ------------------------------------------------------------ графики
def make_plots(res):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    # Рис. 1. Число проб от коэффициента заполнения
    sw = res["sweep"]
    al = [r["alpha"] for r in sw]
    fine = [i / 200 for i in range(10, 195)]
    fig, axes = plt.subplots(1, 2, figsize=(11, 4.3))
    for ax, kind, title in ((axes[0], "found", "Успешный поиск"), (axes[1], "absent", "Неуспешный
    ↪  поиск")):
        ax.plot(al, [r["chain"]["p_" + kind] for r in sw], "o-", label="цепочки (опыт)")
        ax.plot(al, [r["quad"]["p_" + kind] for r in sw], "s-", label="квадр. пробирование (опыт)")
        ax.plot(fine, [theory_probes(a)["chain_" + kind] for a in fine], "--", color="C0", lw=1,
                label="цепочки (теория)")
        ax.plot(fine, [theory_probes(a)["quad_" + kind] for a in fine], "--", color="C1", lw=1,
                label="открытая адресация (теория)")
        ax.set_yscale("log"); ax.set_xlabel("коэффициент заполнения α")
        ax.set_ylabel("среднее число проб"); ax.set_title(title)
        ax.grid(True, which="both", alpha=.3); ax.legend(fontsize=8)
    plt.tight_layout(); plt.savefig("img/lr5_alpha.png", dpi=170); plt.close()

    # Рис. 2. Длины цепочек и распределение Пуассона
    pt = res["poisson"]
    plt.figure(figsize=(6.4, 4.2))
    xs = [r["len"] for r in pt]
    w = 0.4
    plt.bar([x - w / 2 for x in xs], [r["obs"] for r in pt], w, label="опыт")
    plt.bar([x + w / 2 for x in xs], [r["exp"] for r in pt], w, label="Пуассон, λ = α")
    plt.xlabel("длина цепочки"); plt.ylabel("число ячеек")
    plt.title("Распределение длин цепочек (n = 25 000, m = 32 768)")
    plt.legend(); plt.grid(True, axis="y", alpha=.3)
    plt.tight_layout(); plt.savefig("img/lr5_chains.png", dpi=170); plt.close()

    # Рис. 3. B7: время подсчёта частот
```

```
    v7 = res["v7"]
    plt.figure(figsize=(6.8, 4.4))
    sizes = [r["N"] for r in v7["runs"]]
    for name, style in (("список и in", "o-"), ("своя таблица (цепочки)", "s-"),
                        ("своя таблица (квадр.)", "^-"), ("dict", "d-"), ("Counter", "v-")):
        pts = [(r["N"], r["t"][name]) for r in v7["runs"] if name in r["t"]]
        plt.plot(*zip(*pts), style, label=name)
    ext = v7.get("list_extrapolated")
    if ext:
        plt.plot([e["N"] for e in ext], [e["t"] for e in ext], "x--", color="C0",
                 label="список и in (экстраполяция)")
    plt.xscale("log"); plt.yscale("log")
    plt.xlabel("число записей лога N"); plt.ylabel("время, с")
    plt.title("Подсчёт частот клиентов в логе"); plt.grid(True, which="both", alpha=.3)
    plt.legend(fontsize=8); plt.tight_layout(); plt.savefig("img/lr5_v7.png", dpi=170);
↪   plt.close()


# ---------------------------------------------------------------- main
if __name__ == "__main__":
    self_test()
    rng = random.Random(14)
    res = {}

    # ---- 1. Основной эксперимент: n = 25 000, m = 32 768
    keys, found, absent = make_keys(N, rng)
    chain, t_chain = bench_table(lambda: ChainHashTable(M), keys, found, absent)
    quad, t_quad = bench_table(lambda: QuadHashTable(M), keys, found, absent)
    res["main"] = {"alpha": N / M, "chain": chain, "quad": quad,
                   "dict": bench_dict(keys, found, absent)}
    print(f"n = {N}, m = {M}, α = {N / M:.4f}")
    for name, r in (("Цепочки", chain), ("Квадр. пробирование", quad)):
        print(f"{name:<22} вставка {r['t_insert']:.4f} с, поиск {r['t_found']:.4f} с, "
              f"проб: вставка {r['p_insert']:.2f}, найден {r['p_found']:.2f}, не найден
               ↪  {r['p_absent']:.2f}")
    print("dict", res["main"]["dict"])

    # ---- 2. Распределение длин цепочек и пуассоновская модель
    lengths = t_chain.chain_lengths()
    alpha = N / M
    cnt = Counter(lengths)
    res["chains"] = {"max": max(lengths), "empty": cnt[0], "m": M}
    res["poisson"] = [{"len": j, "obs": cnt.get(j, 0),
                       "exp": M * math.exp(-alpha) * alpha ** j / math.factorial(j)}
                      for j in range(max(lengths) + 1)]
    for row in res["poisson"]:
        print(f"  длина {row['len']}: ячеек {row['obs']:>6}, по Пуассону {row['exp']:>9.1f}")

    # ---- 3. Зависимость числа проб от α
    res["sweep"] = []
    for a in (0.1, 0.3, 0.5, 0.7, 0.9, 0.95):
        n = round(a * M)
        k2, f2, a2 = make_keys(n, rng)
        ch, _ = bench_table(lambda: ChainHashTable(M), k2, f2, a2, repeats=1)
        qd, _ = bench_table(lambda: QuadHashTable(M), k2, f2, a2, repeats=1)
        res["sweep"].append({"alpha": a, "n": n, "chain": ch, "quad": qd, "theory":
↪   theory_probes(a)})
        print(f"α = {a}: цепочки {ch['p_found']:.2f}/{ch['p_absent']:.2f}, "
              f"квадр. {qd['p_found']:.2f}/{qd['p_absent']:.2f}")
```

```
# ---- 4. Качественная и неудачная хеш-функции (n = 5000)
nb = 5_000
kb, fb, ab = make_keys(nb, rng)
res["bad"] = {"n": nb, "rows": []}
for hname, hf in (("h = ⌊m·{kA}⌋", hash_mul), ("h = k mod 10", hash_bad)):
    for cname, cls in (("Цепочки", ChainHashTable), ("Квадр. пробирование", QuadHashTable)):
        r, tab = bench_table(lambda: cls(M, hf), kb, fb, ab, repeats=1)
        distinct = len({hf(k, M) for k in kb})
        longest = max(tab.chain_lengths()) if cls is ChainHashTable else None
        res["bad"]["rows"].append({"hash": hname, "method": cname, "distinct": distinct,
                                   "longest": longest, **r})
        print(hname, cname, distinct, longest, {k: round(v, 3) for k, v in r.items()})

# ---- 5. B7: частоты клиентов в логе из 10^6 записей
TOTAL, CLIENTS = 1_000_000, 50_000
log = make_log(TOTAL, CLIENTS, random.Random(7))
sizes = [10_000, 20_000, 50_000, 100_000, 200_000, 500_000, 1_000_000]
LIMIT = 240.0                                   # предельное время одного замера, с
solvers = {"список и in": count_list,
           "своя таблица (цепочки)": lambda lg: count_hash(lg, ChainHashTable, 131_072),
           "своя таблица (квадр.)": lambda lg: count_hash(lg, QuadHashTable, 131_072),
           "dict": count_dict, "Counter": count_counter}
runs, list_skipped = [], False
for size in sizes:
    part = log[:size]
    row, reference = {}, None
    for name, func in solvers.items():
        if name == "список и in" and (list_skipped or (
                runs and "список и in" in runs[-1]["t"] and
                runs[-1]["t"]["список и in"] * (size / runs[-1]["N"]) ** 2 > LIMIT)):
            list_skipped = True
            continue
        t0 = time.perf_counter()
        freq = func(part)
        row[name] = time.perf_counter() - t0
        if reference is None:
            reference = (len(freq), top10(freq))
        else:
            assert (len(freq), top10(freq)) == reference, f"результаты расходятся: {name}"
    runs.append({"N": size, "t": row, "unique": reference[0]})
    print(f"N = {size}: уникальных {reference[0]}, " +
          ", ".join(f"{k} {v:.3f} c" for k, v in row.items()), flush=True)
freq = count_dict(log)
res["v7"] = {"total": TOTAL, "clients": CLIENTS, "unique": len(freq),
             "top10": top10(freq), "runs": runs}
# экстраполяция времени решения через список степенной зависимостью по двум последним точкам
meas = [(r["N"], r["t"]["список и in"]) for r in runs if "список и in" in r["t"]]
if meas[-1][0] < TOTAL:
    (n1, t1), (n2, t2) = meas[-2], meas[-1]
    p = math.log(t2 / t1) / math.log(n2 / n1)
    res["v7"]["list_exponent"] = p
    res["v7"]["list_extrapolated"] = [
        {"N": s, "t": t2 * (s / n2) ** p} for s in sizes if s >= n2]
    print(f"показатель степени {p:.2f}; оценка для N = 10^6: {t2 * (TOTAL / n2) ** p:.0f} c")

with open("res/lr5.json", "w", encoding="utf-8") as f:
    json.dump(res, f, ensure_ascii=False)
make_plots(res)
```

5. Таблицы измерений и графики

5.1. Условия измерений

Параметр	Значение
Операционная система	Linux 6.18 (x86_64), виртуальная машина
Процессор	Intel Xeon, 2,10 ГГц, 1 виртуальное ядро
Оперативная память	4 ГБ
Интерпретатор	CPython 3.12.3
Способ замера времени	`time.perf_counter()`, перед каждым прогоном вызывается `gc.collect()`

Ключи — случайные уникальные целые из $[0;\ 10^7)$. Для основного опыта сгенерированы 10 000 запросов успешного поиска (случайные ключи таблицы) и 10 000 запросов неуспешного поиска (случайные ключи, отсутствующие в таблице). Время — медиана 5 повторов; число проб детерминировано. Для зависимости от α и сравнения хеш-функций использован один повтор, так как важны числа проб, а не время.

Перед замерами выполнена самопроверка: результаты обеих таблиц при случайной последовательности вставок, поисков и удалений совпали с `dict`; последовательность квадратичных проб покрывает все $32\,768$ ячеек.

5.2. Основной эксперимент: $n = 25\,000$, $m = 32\,768$, $\alpha = 0,763$

Таблица 2. Время операций

Реализация	Вставка 25 000 ключей, с	Поиск 10 000 найденных, с	Поиск 10 000 ненайденных, с
Метод цепочек	0,0108	0,0050	0,0040
Квадратичное пробирование	0,0132	0,0054	0,0078
Встроенный `dict` (эталон)	0,0013	0,0009	0,0006

Таблица 3. Среднее число проб (проба — просмотр одной ячейки или элемента цепочки)

Реализация	Проб на вставку	Проб, найден	Проб, не найден
Метод цепочек (опыт)	0,38	1,39	0,76
Метод цепочек (теория: $1 + \alpha/2$; α)	—	1,38	0,76
Квадратичное пробирование (опыт)	2,03	2,07	4,88
Открытая адресация, равномерное хеширование (теория: $\frac{1}{\alpha}\ln\frac{1}{1-\alpha}$; $\frac{1}{1-\alpha}$)	1,89	1,89	4,22

Число проб метода цепочек совпадает с теорией $1 + \alpha/2$ и α с точностью до сотых. Для квадратичного пробирования теория равномерного хеширования даёт 1,89 пробы при успешном и 4,22 при неуспешном поиске против измеренных 2,07 и 4,88: превышение примерно на 10–16 % объясняется вторичной кластеризацией. Среднее число проб при вставке (2,03) близко к $\frac{1}{\alpha}\ln\frac{1}{1-\alpha} \approx 1,89$, так как вставка k-го ключа обходится как неуспешный поиск при заполнении k/m, а усреднение по заполнению от 0 до α даёт именно эту величину.

Время поиска у обеих реализаций почти одинаково (0,0050 и 0,0054 с на 10 000 запросов), хотя квадратичное пробирование делает в полтора раза больше проб: в интерпретаторе Python преобладают накладные расходы вызова метода и вычисления хеша, а не само сравнение. По сравнению с встроенным `dict` реализация цепочками медленнее в 8,5 раза при вставке и в 5,8 раза при поиске, квадратичное пробирование — в 10,4 и 6,2 раза.

5.3. Распределение длин цепочек

Таблица 4. Длины цепочек (метод цепочек) и модель Пуассона

Длина цепочки	Ячеек (опыт)	Ячеек (Пуассон, $\lambda = \alpha$)
0	15 316	15 279,5
1	11 609	11 657,3
2	4 442	4 446,9
3	1 136	1 130,9
4	228	215,7
5	35	32,9
6	2	4,2

Пустых ячеек — 15 316 из 32 768 (46,7 %); по Пуассону ожидается доля $e^{-\alpha} = 46,6$ %. Наибольшая цепочка содержит 6 элементов при средней длине 0,76. Опытное распределение практически совпадает с пуассоновским, то есть метод умножения распределяет ключи так, как если бы хеш-значения были независимыми и равномерными.

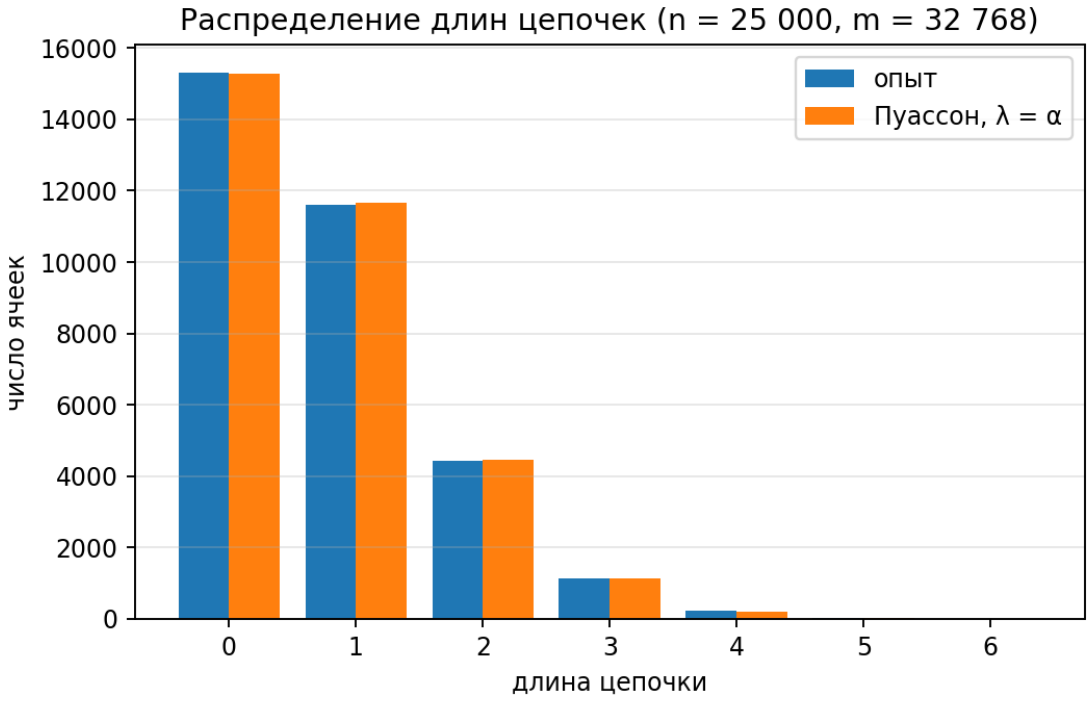


Рис. 1. Распределение длин цепочек и модель Пуассона

5.4. Зависимость числа проб от коэффициента заполнения

Размер таблицы фиксирован ($m = 32\,768$), число ключей $n = \alpha m$.

Таблица 5. Среднее число проб при различных α (опыт / теория)

α	n	Цепочки: найден	теория	Цепочки: не найден	теория	Квадр.: найден	теория	Квадр.: не найден	теория
0,10	3 277	1,05	1,05	0,10	0,10	1,06	1,05	1,11	1,11
0,30	9 830	1,15	1,15	0,29	0,30	1,20	1,19	1,46	1,43
0,50	16 384	1,26	1,25	0,51	0,50	1,45	1,39	2,18	2,00
0,70	22 938	1,36	1,35	0,70	0,70	1,87	1,72	3,82	3,33
0,90	29 491	1,44	1,45	0,89	0,90	2,95	2,56	12,77	10,00
0,95	31 130	1,47	1,48	0,97	0,95	3,50	3,15	22,74	20,00

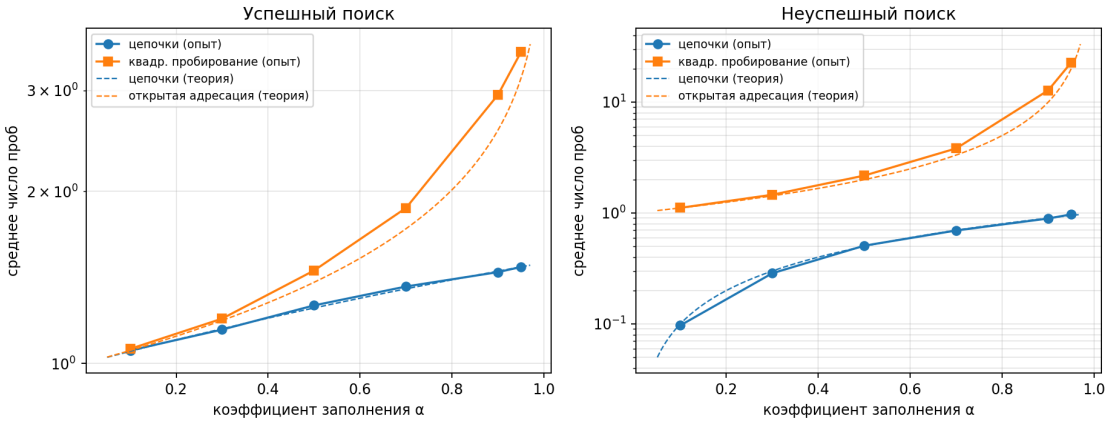


Рис. 2. Число проб как функция α (логарифмическая шкала ординат)

Для метода цепочек число проб растёт линейно по α и остаётся порядка единицы даже при $\alpha = 0{,}95$. При квадратичном пробировании число проб при неуспешном поиске следует кривой $\frac{1}{1-\alpha}$ (с небольшим превышением): от 1,1 при $\alpha = 0{,}1$ и 2,2 при $\alpha = 0{,}5$ до 22,7 при $\alpha = 0{,}95$ — более чем десятикратный рост при увеличении заполнения вдвое (от 0,5 до 0,95).

5.5. Качественная и неудачная хеш-функции

Число ключей уменьшено до $n = 5000$ ($\alpha \approx 0{,}15$): при неудачной функции каждая вставка просматривает сотни элементов, и для $n = 25\,000$ опыт потребовал бы порядка 10^7–10^8 проб на реализацию.

Таблица 6. Сравнение хеш-функций ($n = 5000$, $m = 32\,768$)

Хеш-функция	Метод	Разных значений h (из 5000 ключей)	Макс. цепочка	Проб, найден	Проб, не найден	Вставка 5000, с
h = $\lfloor m\{kA\} \rfloor$	Цепочки	4 640	4	1,08	0,15	0,0020
h = $\lfloor m\{kA\} \rfloor$	Квадр. пробирование	4 640	—	1,09	1,19	0,0024
h = k mod 10	Цепочки	10	545	249,58	500,07	0,0346
h = k mod 10	Квадр. пробирование	10	—	262,34	537,79	0,1129

Неудачная функция сводит 5000 ключей к 10 ячейкам: в методе цепочек длина цепочки достигает 545, среднее число проб успешного поиска возрастает с 1,08 до 250, при квадратичном пробировании — с 1,09 до 262; время вставки 5000 ключей возрастает в 18 раз (цепочки) и в 46 раз (квадратичное пробирование). Причём таблица размером 32 768 при этом почти пуста — проблема не в нехватке места, а в неравномерности h.

6. Результат выполнения дополнительного задания B7

6.1. Постановка

Дан лог из 10^6 записей; запись — идентификатор клиента, совершившего операцию. Требуется определить число уникальных клиентов и 10 клиентов с наибольшим числом операций, а также сравнить время решения с реализацией через список и оператор `in`.

Данные. Лог сгенерирован (генератор с фиксированным зерном): база из 50 000 клиентов с 7-значными идентификаторами, активность клиентов неравномерна — вероятность записи клиента с рангом r пропорциональна $1/r$ (распределение Ципфа), что типично для реальных журналов, где небольшая доля клиентов даёт значительную долю операций. Фактически в логе встретились 47 403 уникальных клиентов.

Способы решения.

1. Список и `in`: два параллельных списка (идентификаторы и счётчики); для каждой записи проверяется `c in seen`, при наличии — `seen.index(c)`, иначе добавление в конец. Оценка $\Theta(N \cdot u)$ при u уникальных клиентах.
2. Собственная хеш-таблица с цепочками, метод `add` увеличивает счётчик.
3. Собственная таблица с квадратичным пробированием.
4. Встроенный `dict`.
5. `collections.Counter`.

Для способов 2–3 размер таблицы увеличен до $m = 131\,072$ ($\alpha \approx 0{,}36$ для $u = 47\ 403$), так как u превышает ёмкость таблицы из задания варианта ($m = 32\,768$), а открытая адресация требует $\alpha < 1$. Все пять способов дают один и тот же набор частот; при равенстве частот клиенты упорядочиваются по возрастанию идентификатора, и проверка совпадения числа уникальных клиентов и списка из 10 лидеров выполняется в программе (`assert`) на каждом размере.

6.2. Результаты

Таблица 7. Время подсчёта частот, с

N	Уникальных клиентов u	список и `in`	своя таблица (цепочки)	своя таблица (квадр.)	`dict`	`Counter`
10 000	3 850	0,063	0,055	0,010	0,001	0,000
20 000	6 635	0,218	0,057	0,016	0,001	0,001
50 000	12 668	0,855	0,051	0,027	0,003	0,002
100 000	19 701	2,506	0,086	0,047	0,006	0,005
200 000	28 701	10,016	0,114	0,101	0,011	0,008
500 000	41 113	37,228	0,220	0,205	0,029	0,021
1 000 000	47 403	93,477	0,386	0,410	0,065	0,044

Измерения выполнены на префиксах лога; последняя строка — полный лог из 10^6 записей (все значения измерены, экстраполяция не применялась). Все значения — однократные замеры. Постоянная составляющая около 0,05 с у цепочек при малых N — создание 131 072 пустых списков (время не меняется от 10 000 до 50 000 записей).

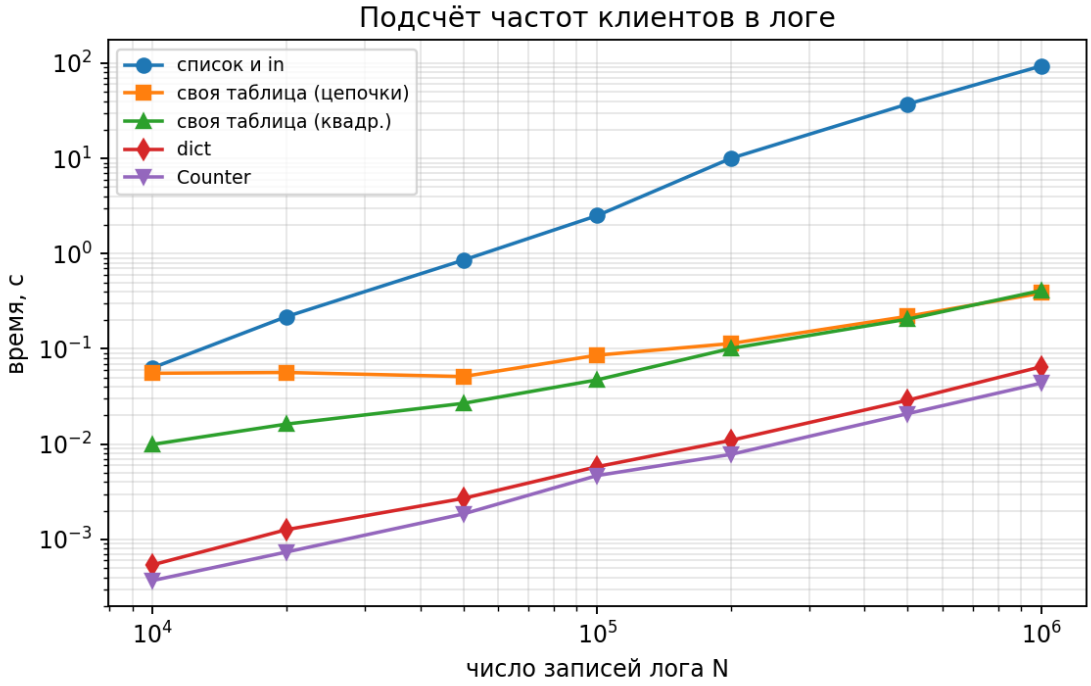


Рис. 3. Время подсчёта частот в зависимости от числа записей

Таблица 8. Проверка оценки $\Theta(N \cdot u)$ для решения через список

N	$N \cdot u$	Время списка, с	Время / ($N \cdot u$), нс	Время списка / время `dict`
10 000	38 500 000	0,063	1,64	117
20 000	132 700 000	0,218	1,64	172
50 000	633 400 000	0,855	1,35	317
100 000	1 970 100 000	2,506	1,27	432
200 000	5 740 200 000	10,016	1,74	914
500 000	20 556 500 000	37,228	1,81	1 288
1 000 000	47 403 000 000	93,477	1,97	1 436

Отношение времени к $N \cdot u$ остаётся в пределах 1,3–2,0 нс, то есть время пропорционально числу пар «запись — уже известный клиент», что подтверждает оценку $\Theta(N \cdot u)$. Так как число уникальных клиентов u само растёт вместе с N, зависимость от N получается сверхлинейной (между $N = 10^5$ и 10^6 время выросло в 37 раз при росте N в 10 раз).

Результат на полном логе: число уникальных клиентов — 47 403. Десять клиентов с наибольшим числом операций:

Таблица 9. Топ-10 клиентов по числу операций

Место	Идентификатор клиента	Число операций	Доля лога, %
1	6433012	87 392	8,7
2	3530829	44 212	4,4
3	7624039	29 290	2,9
4	1810111	22 101	2,2
5	2215279	17 455	1,7
6	9990608	14 702	1,5
7	2579240	12 637	1,3
8	7135241	11 030	1,1
9	1973060	9 778	1,0
10	9513358	8 767	0,9

Клиент-лидер совершил 87 392 операций (8,7 % лога), десять лидеров — 25,7 % всех записей.

6.3. Анализ

На полном логе решение через список заняло 93,5 с, тогда как `dict` — 0,065 с (быстрее в 1 436 раз), `Counter` — 0,044 с (в 2 134 раз), собственные таблицы — 0,386 с (цепочки) и 0,410 с (квадратичное пробирование). Разрыв растёт с N: решение через список имеет сверхлинейный рост, хеш-решения — близкий к линейному по N ($\Theta(N)$ в среднем). Собственные таблицы уступают `dict` примерно в 6 раз (а `Counter` — в 9), так как написаны на Python, но сохраняют принципиально верную асимптотику; между собой цепочки и квадратичное пробирование при $\alpha \approx 0{,}36$ практически не различаются.

Для массива из 10^6 записей использование списка и `in` практически неприменимо: время решения у списка превышает минуту, и оно растёт быстрее, чем N. Заметим, что распределение Ципфа в данном опыте смягчает оценку: частые клиенты быстро попадают в начало списка и находятся быстро; при более равномерной активности клиентов можно ожидать ещё большего разрыва.

7. Выводы

1. Хеш-таблица с методом умножения и $m = 2^{15}$ обеспечивает поиск за малое число проб: при $\alpha = 0,763$ метод цепочек выполняет в среднем 1,39 пробы для найденного ключа и 0,76 для ненайденного, что совпадает с теорией $1 + \alpha/2$ и α.
2. Квадратичное пробирование с $c_1 = c_2 = 1/2$ корректно для $m = 2^s$ (проходит все ячейки), но требует $\alpha < 1$ и деградирует вблизи заполнения: число проб неуспешного поиска растёт по закону, близкому к $\frac{1}{1-\alpha}$, от 1,1 при $\alpha = 0{,}1$ до 22,7 при $\alpha = 0{,}95$. Метод цепочек устойчив к росту α (0,97 пробы при $\alpha = 0{,}95$). Измеренные значения для квадратичного пробирования на 10–16 % превышают оценки равномерного хеширования из-за вторичной кластеризации.

3. Распределение длин цепочек близко к пуассоновскому: при средней длине 0,76 около 47 % ячеек пусты, максимальная цепочка содержит 6 элементов. Различие между средним и худшим случаем — типичная черта хеширования.
4. Качество хеш-функции критично: при $h(k) = k \bmod 10$ число проб возрастает со значений порядка единицы до сотен, а трудоёмкость поиска вырождается в $\Theta(n)$.
5. Встроенный `dict` в 6–10 раз быстрее собственных реализаций, так как написан на C; собственные таблицы имеют учебное назначение и позволяют наблюдать число проб и длины цепочек.
6. Прикладная задача B7 показывает принципиальный выигрыш хеширования: подсчёт частот в логе из 10^6 записей через хеш-таблицу занимает доли секунды, через список и `in` — более полутора минут ($\Theta(N \cdot u)$ против $\Theta(N)$). Хеш-таблица не сохраняет упорядоченность ключей, поэтому топ-10 получен отдельным проходом по найденным частотам.

8. Ответы на контрольные вопросы

1. Почему прямая адресация неприменима к таким ключам, как ИНН или номер договора? Прямая адресация требует массива с одной ячейкой на каждое возможное значение ключа. Для 12-значного ИНН это 10^{12} ячеек при реально хранимых тысячах записей; номера договоров могут включать буквы и разделители, и множество допустимых значений ещё больше. Память расходуется на почти пустую таблицу. Хеш-функция сжимает пространство ключей до таблицы размера m (в варианте 14 — 2^{15}).

2. Дайте определение коллизии. Почему коллизии неизбежны? Коллизия — ситуация $h(k_1) = h(k_2)$ при $k_1 \neq k_2$. Так как число возможных ключей $|U|$ намного больше m, отображение $U \to \{0, \dots, m-1\}$ не может быть взаимно однозначным (принцип Дирихле). Кроме того, они возникают задолго до заполнения таблицы (парадокс дней рождения). В нашем опыте при $\alpha = 0{,}76$ в ячейках с коллизиями оказалось 5843 ячейки, содержащие 13 391 из 25 000 ключей (53,6 %), а наибольшая цепочка содержит 6 элементов.

3. Перечислите требования к хеш-функции. Почему $h(k) = k \bmod 10$ плоха? Быстрое вычисление; равномерное распределение значений по ячейкам; зависимость от всех разрядов ключа. Функция $k \bmod 10$ принимает лишь 10 значений и зависит только от последней цифры, поэтому все ключи концентрируются в 10 ячейках. В опыте ($n = 5000$) при этом наибольшая цепочка содержит 545 элементов, число проб успешного поиска — 250 против 1,08 у метода умножения (табл. 6).

4. Почему в методе деления размер таблицы рекомендуется брать простым числом? При $h(k) = k \bmod m$ с $m = 2^s$ учитываются только младшие s бит ключа, а остальные разряды игнорируются; если ключи имеют закономерность младших разрядов (например, кратны степени двойки или 10), они попадают в малое число ячеек. Если m имеет общий делитель с шагом закономерности, то используется лишь часть ячеек. Простое m не имеет общих делителей с типичными закономерностями, и в результат вносят вклад все разряды. Метод умножения (вариант 14) этого требования не предъявляет, поэтому допустимо $m = 2^{15}$.

5. Что такое коэффициент заполнения? Какие значения он может принимать при цепочках и при открытой адресации? $\alpha = n/m$ — отношение числа хранимых ключей к размеру таблицы. При цепочках α может быть любым неотрицательным числом, в том числе больше 1 (это средняя длина цепочки): например, в задании B7 при $m = 32\,768$ и 47 403 клиентах было бы $\alpha \approx 1{,}45$. При открытой адресации все элементы лежат в массиве, поэтому $\alpha < 1$ (при $\alpha = 1$ вставка нового ключа невозможна), а эффективная работа — только при заметно меньших значениях; в варианте 14 $\alpha = 0{,}763$.

6. Объясните формулу $C_{\text{ср}} \approx \frac{1}{1-\alpha}$. Что происходит при $\alpha \to 1$? При равномерном хешировании вероятность того, что очередная проба попадёт в занятую ячейку, равна α. Число проб неуспешного поиска — геометрическая величина: проб будет не меньше $i+1$ с вероятностью α^i, откуда $C = \sum_{i \geq 0} \alpha^i = \frac{1}{1-\alpha}$. При $\alpha \to 1$ значение неограниченно растёт: опытное число проб при квадратичном пробировании составило 2,2 при $\alpha = 0{,}5$ и 22,7 при $\alpha = 0{,}95$ (табл. 5). Поэтому таблицу расширяют заранее.

7. Что такое первичная кластеризация и как с ней борются? При линейном

пробировании $h_i = (h + i) \bmod m$ занятые ячейки образуют сплошные участки; ключ, попавший в любую ячейку участка, продлевает его, и растущие участки поглощают друг друга. Бороться можно квадратичным пробированием (в нашей работе: $h + i(i + 1)/2$), двойным хешированием, снижением α (рехешированием) и качественной хеш-функцией. Квадратичное пробирование устраняет первичную кластеризацию, но оставляет вторичную: ключи с одинаковым $h(k)$ идут по одной последовательности (в опыте это дало 4,88 пробы против 4,22 теоретических).

8. Почему при открытой адресации нельзя просто обнулять ячейку при удалении? Поиск ключа заканчивается на первой пустой ячейке последовательности. Если обнулить ячейку, через которую проходила последовательность проб другого ключа, поиск этого ключа остановится на ней и неверно сообщит об отсутствии. Поэтому в ячейку записывают пометку «удалено» (tombstone): поиск через неё продолжается, вставка может её использовать. В нашей реализации так и сделано, а самопроверка со случайными удалениями совпала со словарём `dict`.

9. Что такое рехеширование? Почему вставка остаётся $O(1)$ амортизированно? Когда α превышает порог (обычно 0,75), создаётся таблица большего размера (обычно вдвое), и все элементы вставляются в неё заново с новой хеш-функцией; эта операция стоит $\Theta(n)$. Если размер удваивается, то суммарные затраты на перераспределение при росте до n элементов составляют $n + n/2 + n/4 + ... < 2n$, то есть в среднем $O(1)$ на вставку. В варианте 14 $\alpha = 0{,}763$ уже превышает типичный порог 0,75, и на практике такую таблицу следовало бы расширить.

10. Какие запросы к данным хеш-таблица выполнять не умеет? Какую структуру следует выбрать в этом случае? Хеш-таблица не хранит порядка ключей, поэтому не поддерживает диапазонные запросы («суммы от 100 000 до 500 000»), поиск минимума и максимума, соседних ключей и вывод по возрастанию. В таких случаях выбирают упорядоченные структуры — сбалансированное дерево поиска (красно-чёрное, B-дерево) или упорядоченный массив с двоичным поиском. В задании B7 для вывода топ-10 потребовался отдельный проход по построенным частотам.